

AgentForge Project Details and Question and Answer Guide

Technical and functional reference for the BOC 2.0 presentation

Prepared 10 October 2026 | Application folder my-app | Code baseline c323108

This guide explains the implemented support platform, its user roles, technical architecture, data and integrations. It includes 40 presentation questions with suggested answers and distinguishes current functionality from future development.

Contents

1 Project overview 3

2 Users and responsibilities 4

3 Product areas and implemented features 4

4 Actual system architecture 5

5 Technology stack 7

6 Codebase organisation 7

7 How authentication and identity work 8

8 The shared chat workflow 9

9 Company documents and retrieval augmented generation 10

10 Firebase connections and collection access 11

11 Customer database sync 12

12 Stored data and ownership 13

13 Website widget integration 14

14 API reference 15

15 Pages and navigation 17

16 Runtime configuration and deployment 18

17 Implemented capabilities and honest boundaries 20

18 Security and engineering limitations to explain accurately 21

19 Local evidence from this review 22

20 Design decisions and tradeoffs 22

21 Question and answer preparation 23

22 Suggested presentation walkthrough 28

23 Glossary 29

24 Source reference map 29

1 Project overview

AgentForge is a prototype platform for adding company-specific AI support to a business website. A company can use a hosted chat widget or build its own interface and call the enterprise chat API. The assistant answers questions using uploaded company documents, retrieves a signed-in customer's orders, sends an email when requested, and creates a manager review ticket for qualifying refund requests.

The application includes three account areas: platform operations, company administration, and a customer portal. Two fictional storefronts demonstrate the integration: Walkwave, a Sri Lankan shoe retailer, and Nova Electronics, a technology retailer. Product catalogues, prices, customers, tracking references, and orders in these demonstrations are sample business data. There is no checkout or payment execution.

The most accurate description is a working support integration prototype with company administration. The wider idea of an autonomous enterprise operations control plane appears in earlier material and infrastructure helpers, but the active application is narrower. Its current support workflow runs in Next.js with Firestore, Gemini, Pinecone, and SMTP.

A short introduction for the presentation

"AgentForge helps a company add AI customer support to its website through a ready-made chat widget or its own interface. The assistant uses the company's uploaded policies, checks the authenticated customer's order records, sends requested updates by email, and hands qualifying refund requests to a manager. Company admins manage knowledge, customer access, database connections, website settings, and API keys from one portal."

The problem being addressed

Customers often need answers scattered across policy documents, order databases, and support teams. A general chatbot can answer conversationally but may lack the business context and permission to read private records. AgentForge connects those sources behind a company identity and customer identity, while leaving decisions such as high-value refund approval to a human.

For a company, the value is a shared support backend that can serve a website bubble or a custom application. For customers, the value is access to policy answers and personal order information without navigating separate systems. Actual reductions in support workload, response time, or cost have not been measured in this codebase.

2 Users and responsibilities

Role	Entry point	Main responsibilities
Platform operator	/ops/login	Create company admin accounts, enable or disable admins, inspect platform activity and summary metrics
Company admin	/admin/login	Manage customer accounts, knowledge documents, API keys, Firebase settings, website widget settings, sync, and review tickets
Portal customer	/portal/login	Ask the assistant questions and view saved conversation history
Company website customer	Walkwave, Nova, or an integrated client website	Ask public policy questions and, with verified identity, retrieve personal orders or request actions
Anonymous website visitor	Installed widget	Ask policy questions; personal orders, refund review, and email workflows require customer identity
Company backend developer	Enterprise API and widget session API	Keep the company API key on the server, verify the client's login, and map the customer to an AgentForge user ID

Each company is represented by a tenant ID. Tenant IDs connect admins, users, documents, actions, API keys, and settings. The operations role has platform-wide visibility, while the company admin and customer flows generally operate within one tenant.

The same browser profile uses one platform session cookie. Selecting a demo customer changes that browser's signed-in account. During a presentation, use separate browser profiles for the manager and the customer so that one demonstration does not replace the other session.

3 Product areas and implemented features

Company administration

The dashboard displays customer counts, document counts, saved chat counts, and escalation counts from Firestore. Users and Access lists customer identities, allows an admin to create local users, and enables or disables their access. Imported company customers show both their AgentForge user ID and their original client customer ID.

Knowledge Base supports uploading and deleting PDF or TXT policy documents. API Keys supports creating, viewing the key prefix, copying a newly generated key, and revoking a key. Website Agent and Activity shows the company website integration and recorded database, email, and report actions.

Company Settings has three separate configuration areas: website widget configuration, company Firebase connection and collection access, and customer database sync. Escalations lets an admin inspect the original customer request, review reason, identity, status, and optional decision note, then approve or reject a pending request. Reports downloads recorded usage and manager review history as CSV. Analytics displays saved customer chats over the last seven days using Asia/Colombo dates.

Platform operations

The operator can create a business admin account with a generated tenant ID, list business admins, and toggle their active status. Global Agent Analytics reads recorded actions across tenants. Platform totals read Firestore records, but several charts and infrastructure indicators are simulated. The operations report page is a static presentation screen rather than a completed reporting subsystem.

Customer portal

Customers can request an account using a tenant ID, email, name, and password. Self-registered users are created inactive and require admin activation. Locally managed active users can log in, use the chat page, and view their saved history. Customers imported from a company database must use the company's own login system; their passwords are not imported into AgentForge.

Website demonstrations

Walkwave uses a React chat panel inside its storefront. Nova installs the reusable JavaScript widget and also offers a custom enterprise API interface. Walkwave includes shoe category filtering and product detail modals. Nova displays a sample electronics catalogue. These are demonstration storefronts, not commerce implementations.

4 Actual system architecture

The browser renders React pages or loads the reusable widget script. Requests reach Next.js route handlers. A cookie session, API key, or widget token identifies the caller. The route validates access and calls a shared chat service. That service decides whether to perform an order lookup, generate a grounded policy answer, send an email, or create a manager review ticket.

The main request paths are:

- Portal and Walkwave: browser -> /api/chat -> shared chat service -> Firestore or policy retrieval -> server-sent events -> chat panel.
- Nova custom interface: browser -> /api/demo/enterprise-chat -> server-side call to /api/v1/chat -> shared chat service -> JSON response.
- Reusable widget: host website -> widget.js -> hosted iframe -> /api/widget/chat with a short-lived token -> shared chat service -> JSON response.
- Document upload: admin -> /api/admin/documents -> text extraction -> Gemini embedding -> Pinecone vectors -> Firestore document record.
- Customer sync: admin settings or Users and Access -> customer sync service -> external company Firestore -> platform user profiles.
- Manager review: customer chat -> escalation record -> admin decision -> customer polling -> displayed status.

This is a single Next.js application with shared libraries and route handlers. It is not a deployed collection of independently running agents, microservices, or execution sandboxes. Labels such as Database Agent and Email Agent describe application capabilities. In the active support path, the server makes the routing decisions and calls the associated functions.

Services used by the active support workflow

Service	Purpose	Important boundary
Firestore	Platform records, chat history, settings, actions, review tickets, and company orders	Application access checks and deployed Firestore permissions must both be correct
Gemini API	Embeddings and policy answer generation	Receives the selected company knowledge and conversational input
Pinecone	Similarity search over policy document chunks	Queries include a tenant metadata filter
SMTP	Requested email delivery	SMTP acceptance is distinct from arrival in the recipient's inbox
Next.js runtime	Pages, authentication endpoints, integration APIs, and coordination	Hosts the business logic and shared chat service

AWS SQS, CloudWatch Logs, DynamoDB, and SSM helpers exist in the repository. The active chat path does not call them. Their existence should be presented as supporting code or future integration work, not proof that every request passes through a live hybrid-cloud audit pipeline.

5 Technology stack

Technology	Version declared or installed	Usage
Next.js	16.3.5	App Router, React pages, route handlers, production build, and middleware
React and React DOM	19.2.8	UI state, effects, components, forms, and chat rendering
TypeScript	Dependency range ^5	Typed application source with strict compiler settings
Tailwind CSS	Dependency range ^4	Layout, colours, spacing, responsive styles, and utility classes
Firebase JavaScript SDK	12.19.0 installed	Firestore access and initialized Auth and Storage clients
Google Generative AI SDK	0.24.1	Gemini generation and embedding requests
Pinecone SDK	9.0.0	Vector upsert, metadata-filtered search, and deletion
jose	Dependency range ^6.2.12	JWT creation and verification
bcryptjs	Dependency range ^3.0.3	Password hashing and comparison
pdf-parse	2.4.5	Selectable-text PDF extraction
jsPDF	Dependency range ^4.2.1	PDF generation utility and seeded policy assets
Nodemailer	Dependency range ^10.0.16	SMTP email sending
Recharts	Dependency range ^3.10.1	Admin chat activity chart
AWS SDK packages	Dependency range ^3.1141.0	Optional infrastructure helper modules

The active text model defaults to gemini-flash-lite-latest and can be overridden through GEMINI_MODEL. Embeddings use gemini-embedding-2. The code slices embedding vectors to 768 values, so the connected Pinecone index must be compatible with that shape. These are configured model names in the source; availability and output behaviour depend on the connected provider account.

6 Codebase organisation

The reviewed source inventory contained 148 JavaScript and TypeScript files, approximately 10,511 lines, 29 application page components, 42 API route files, 58 exported API handlers, and 19 shared library files. Generated .next output and node_modules are excluded from those counts.

Location	What it contains
app/(public)	Landing page, integration guide, architecture explanation, security information, pricing placeholder, and login chooser

Location	What it contains
app/admin	Company dashboard, access management, documents, keys, settings, activity, analytics, reports, and escalation review
app/ops	Platform administration and overview screens
app/portal	Customer registration, login, chat, and history
app/api	HTTP handlers for account operations, administration, chat, integrations, and legacy features
app/components	Authentication context, shared navigation, storefront components, and chat panels
lib	Chat coordination, AI retrieval, tools, sessions, widget tokens, company database access, customer sync, and optional AWS helpers
public/widget.js	Script installed on a client's website
public/widget	Hosted iframe JavaScript and CSS
public/demo	Sample company policy PDFs and TXT files
scripts	Demo setup, verification scripts, embedding experiments, and AWS resource setup
_legacy_routes	Earlier studio, launch, replay, incident, and escalation pages outside the active App Router tree

Files such as migrate.js, extract-layout.js, add-fonts.js, and fix-*.js are development conversion or repair utilities. They are not part of the normal request workflow. Several depend on earlier HTML files or page locations. They should not be run as routine deployment steps.

7 How authentication and identity work

Platform cookie sessions

Locally managed users and business admins authenticate by matching their email against Firestore and comparing their password with a bcrypt hash. Active status is required at login. Operators authenticate using configured environment credentials, with hardcoded defaults if those settings are absent.

Successful login creates an HS256 JWT with a 24-hour expiry. The token is stored in an HTTP-only session cookie. The cookie uses SameSite=Lax and is marked Secure in production. Middleware redirects unauthenticated or incorrect-role visitors from the protected operations, administration, and customer chat/history pages.

The application uses its own cookie authentication. Although Firebase Auth is initialized in lib/firebase.ts, the active password login endpoints do not authenticate those sessions through Firebase Auth. An AgentForge JWT therefore should not be described as a Firebase Auth identity.

Enterprise API keys

A company admin can generate an `af_` key using 24 cryptographically random bytes. The key is stored in Firestore with the tenant ID and active status. The enterprise chat API checks the Bearer key and derives the tenant from the key record, rather than trusting a tenant ID sent by a customer.

A trusted company backend can supply a `customerId`. The API checks that this user exists, belongs to the key's tenant, and is active. This customer ID is the AgentForge platform user ID. For imported customers, order lookup then maps that identity back to the original client customer ID.

Keys are currently stored as plaintext and returned in the admin list response. The UI says the new key will only be shown once, but the backend does not enforce one-time visibility. Key hashing, stronger management permissions, rotation procedures, and usage limits remain production hardening work.

Widget sessions

Personal widget sessions are created by the client's backend using its secret company API key. That backend must first verify the customer's own login and map the customer to an active AgentForge profile. The token endpoint on the company website should not accept an arbitrary customer ID supplied by the browser.

Widget JWTs expire after 30 minutes. They include the company, allowed website origin, optional customer identity, issuer, audience, and a unique token identifier. Verification rechecks the widget settings and customer active status. An anonymous bootstrap session contains no customer identity and cannot access private orders.

Website origin checks, iframe frame-ancestors policy, and `postMessage` source/origin checks support safe embedding. A website origin is a browser integration boundary, not a substitute for customer authentication. Personal access depends on the verified token.

8 The shared chat workflow

All three main delivery channels use `lib/chat-service.ts`. Input validation requires a non-empty message no longer than 4,000 characters. It keeps at most eight recent history items, each bounded to 4,000 characters, and generates a request ID if a suitable ID is not supplied.

Step 1 Check for refund review

Refund or reimbursement messages are examined by a server-side rule. For the demo companies, amounts above LKR 50,000 require review. Foreign-currency refund requests and refund messages mentioning a manager, human, or CEO also trigger review. A signed-in customer is required.

The service creates a pending escalation containing the tenant, customer, original request, review reason, rule, and timestamp. A transaction prevents another ticket from being created for the same tenant, customer, and request ID. The reply explicitly says the request is awaiting manager review and no refund has been issued.

The rule only runs when the message contains `refund` or `reimburse`. A general message such as "I need a human" does not automatically produce a ticket. This is an important distinction when describing the current handoff capability.

Step 2 Identify order and email intent

The service uses keyword and regular-expression checks for order, shipping, tracking, and email requests. Specific order IDs are recognized in `WW-1234` or `NV-1234` format. General order requests retrieve the customer's available orders, up to ten records.

The order answer is assembled from database fields, not invented by the language model. It reports product, order ID, shipping status, tracking reference, and estimated delivery when present. Missing tracking or dates are acknowledged.

Step 3 Answer policy questions

Messages that do not enter the order path retrieve indexed company knowledge. If there is no indexed text, the service returns an honest message asking for a policy upload. With knowledge available, it calls Gemini with the current message, bounded history, and selected company text.

The policy generation call disables tools and instructs the model to answer from the provided knowledge. It must not invent orders, prices, policies, or completed actions. Source titles are returned with the result. The titles identify retrieved documents, but they are not page-level quotations or proof that every generated sentence is correct.

Step 4 Send an explicitly requested email

The current message must express email intent. The recipient is an email address explicitly supplied in that message, or the profile email, or the configured demonstration inbox for demo accounts. The service checks the user and SMTP configuration before sending.

A Firestore transaction claims the tenant/customer/request ID before SMTP execution. A repeated request ID does not send again. Success means the SMTP server accepted the email; the assistant does not claim guaranteed inbox arrival. Failed requests are recorded as failed, and a deliberate retry requires a new request ID.

Step 5 Record and return the result

The shared result includes reply, actions, sources, and optional orders or escalationId. Enterprise and widget endpoints add requestId to their JSON response. The portal endpoint emits agent, text, and result events through server-sent events.

The server-sent event interface provides action notifications and a completed answer. It does not stream individual Gemini tokens. Chat history is saved for identified customers after the normal result path. Several early-return paths, such as no matching orders or missing knowledge, return before history persistence.

9 Company documents and retrieval augmented generation

Retrieval augmented generation, or RAG, means selecting relevant stored business information and supplying it to the language model before it writes an answer. Here, the business information is company policy text.

Uploads must be a non-empty PDF or TXT no larger than 4 MB. TXT is decoded directly. PDF extraction uses pdf-parse version 2 with worker and canvas support. Scanned PDFs without selectable text require OCR, which this prototype does not provide. The extracted document text is limited to 80,000 characters.

Text is divided into chunks of up to 1,300 characters, advancing by 1,100 characters. Consecutive full chunks therefore overlap by 200 characters. Each chunk is embedded through Gemini and stored in Pinecone with tenantId, docId, title, and content metadata. The document is marked indexed in Firestore only after vector creation and upsert succeed.

At retrieval time, the service first finds the tenant's Firestore documents whose status is indexed. It then embeds the question and searches Pinecone for five matches with a tenant filter. Only vectors whose docId still

exists in the indexed document set are accepted. This prevents an orphan vector from restoring a document that is no longer active.

If vector search is unavailable or produces no usable matches, the service falls back to actual indexed document text stored in Firestore. It selects recent documents with a total text budget of 20,000 characters. This fallback is still based on persisted company content; it is not a hardcoded invented policy.

Document deletion checks tenant ownership, removes matching Pinecone vectors when configured, and deletes the Firestore document. The original uploaded binary is not stored by this upload path. Firestore retains the extracted text and document metadata.

10 Firebase connections and collection access

Company Settings accepts a Firebase web configuration object. The server requires a `projectId` and `apiKey` when a configuration is supplied. An empty configuration uses the platform Firestore database. The external connection is initialized with a tenant-specific application name containing a fingerprint of the configuration, allowing separate project configurations to coexist.

The selected database is the default Firestore database for the Firebase app. The settings are collection-level controls, not a selector for multiple named Firestore databases. The application does not provide a Realtime Database connector, SQL execution, or automatic collection discovery.

Allowed collections

Admins enter exact top-level collection names separated by commas or new lines. The list is normalized, deduplicated, and checked on the server. Names can contain letters, numbers, underscores, and hyphens; paths and wildcards are rejected. The limit is 50 names, each at most 100 characters.

Order lookup and customer sync enforce this list. An explicitly empty list blocks company collection reads through those integrations. If a tenant has never stored `allowedCollections`, the legacy default allows only `demo_orders`. The default order collection is `demo_orders`, but admins can set another valid top-level collection name.

Listing a collection does not create a generic query capability. The active assistant can read its configured order collection using the supported schema, while customer sync can read its configured profile source. Other collections need a supported integration. Platform account records, settings, logs, and review tickets are managed separately from this company data allowlist.

The allowlist limits what these application readers request. It does not rewrite Firestore security rules or prevent direct access allowed by the Firebase project's deployed rules. Firebase web configuration is not a service-account credential and does not itself grant privileged backend access.

Order ownership checks

Before reading orders, the service verifies the customer record belongs to the tenant and is active. It queries using both `tenantId` and `customerId`. For imported customers, `customerId` is the original external customer ID. It also checks that the external project recorded on the customer still matches the configured order database project.

Order records require `tenantId` and `customerId` for ownership filtering, along with `orderId`, `productName`, `shippingStatus`, and optional `tracking`, `delivery`, `amount`, and `currency` fields. There is no configurable field mapping. A company whose database uses different names must adapt its schema or extend the integration.

Schema notes in Company Settings are documentation for admins. They do not change the query structure, grant access, or automatically teach the model new database fields.

11 Customer database sync

Sync imports selected customer profile fields from the connected company's Firestore database into platform users. It requires the external Firebase connection, a valid source collection, and an allowed collection entry. A company filter can be configured as a field and value for shared collections. Both filter values must be supplied together.

The platform project cannot import its own users collection as a source. If the source is the platform project, a company filter is required. For a dedicated external collection, the filter may be omitted; records carrying a different tenantId are still skipped in that mode.

Sync accepts name, fullName, or displayName as the name, plus email and selected activity/timestamp fields. Passwords, password hashes, and unrelated private profile fields are not imported. Source document IDs must follow the supported identifier pattern.

A deterministic hash of the source identity creates a stable AgentForge user ID. The record stores the original externalCustomerId, externalProjectId, externalCollection, source key, source activity, and admin disable state. Prepared demo profiles can be adopted under limited matching conditions.

The sync service supports up to 500 source customers per run. It requests 501 to detect an oversized source. A transaction acquires a two-minute lock, and successful automatic sync runs are throttled for approximately 30 seconds. Platform writes are committed in batches of up to 450 records.

Users and Access refreshes its list every ten seconds while visible, but the sync service's throttle normally limits source reads to about once every 30 seconds. Sync now can request a forced run. There is no scheduled background sync worker or source-database realtime listener.

Missing or inactive source profiles lose effective access after a successful sync. An admin disable remains effective after subsequent imports. A source failure leaves previously imported profiles visible and reports an error; it does not automatically revoke all previous access. A source change creates a new source identity, and old imported identities are disabled during successful reconciliation.

Turning sync off stops reconciliation. It does not immediately disable every previously imported account. These are important lifecycle considerations for production use.

12 Stored data and ownership

Collection	Purpose	Key information
business_admins	Company admin accounts	Email, name, company, tenantId, passwordHash, active status, creation time
users	Platform customer identities	Tenant, profile, local password hash when applicable, active state, external identity mapping
tenant_settings	Company configuration	Firebase configuration, allowed collections, order collection, schema notes, widget configuration, customer sync settings
api_keys	Company integration credentials	Key value, tenantId, creator, active state, usage count, last used time
documents	Indexed company policies	Tenant, title, filename, extracted text, chunk count, index status, creation time
chat_history	Saved identified customer conversations	Tenant, user, original message, reply, actions, timestamp
agent_actions	Recorded tool activity	Tenant, user, agent type, action details, success or error, timestamp
escalations	Manager review queue	Tenant, customer, original request, reason, rule, status, admin decision, note, timestamps
demo_email_requests	Email duplicate prevention	Tenant/customer/request ID, pending/sent/failed status, creation time
customer_sync_state	Sync coordination and status	Lock owner/expiry, attempt time, completion time, counters, status, error
widget_limits	Per-company widget request limits	Hashed company/operation/IP key, count, reset time
Configured order collection	Company-owned orders	Tenant and customer ownership, product, order status, tracking, delivery, amount, currency
reports	Generated report utility output	Tenant, user, topic, PDF in base64, creation time

Legacy code also references agents, traces, trace_summaries, customers, sales, fleet_nodes, and api_logs. Some are used only by older APIs or seeding routines. Their presence does not prove the active assistant writes complete decision traces.

Pinecone stores policy vectors and metadata separately from Firestore. There is no repository-defined general retention schedule, backup workflow, data export/deletion lifecycle, or formal privacy compliance implementation. Raw identified customer messages may be stored in history and review tickets.

13 Website widget integration

An admin sets the assistant name, brand colour, enabled state, and allowed website origins. Up to 20 origins are supported. Production origins must use HTTPS without a path, query, or fragment. Localhost and loopback origins may use HTTP for testing. Separate www and non-www origins must be listed separately.

The generated script contains the AgentForge service URL and public company ID. A personal integration adds data-token-url, pointing to a first-party endpoint on the client's own website. The secret company API key must remain on that website's backend.

widget.js creates a launcher using Shadow DOM and opens a hosted iframe. Shadow DOM reduces style collisions with the host website. The iframe has a sandbox allowing scripts and same-origin operation; its content security policy restricts framing to the saved origins. Text messages are rendered with `textContent` rather than interpreted as HTML.

The script checks both message origin and message source for `postMessage` communication. It can open, close, refresh identity, and destroy the widget through `window.AgentForgeWidget`. Changing identity clears the previous conversation and ticket polling state. The widget avoids depending on third-party platform session cookies by using its Bearer token inside the iframe.

Widget sessions expire after 30 minutes. There is no silent refresh timer; reopening or explicitly refreshing identity obtains a new session. The widget chat endpoint allows 30 requests per company/operation/IP minute. Anonymous bootstrap allows 20 per minute, and personal session creation allows 30. These limits do not apply to every other chat or login endpoint.

Client websites with their own content security policy must permit the service's script, configuration connection, and iframe. The client also needs to accommodate the widget's injected styling. These are deployment requirements, not settings that AgentForge can automatically change on another website.

14 API reference

Authentication and core support

Endpoint	Methods	Contract
/api/auth/ops-login	POST	Operator environment credentials -> platform session
/api/auth/admin-login	POST	Active business admin email/password -> company session
/api/auth/user-login	POST	Active local customer credentials -> customer session
/api/auth/user-register	POST	Profile, password, and tenant -> inactive pending account
/api/auth/session	GET	Current cookie session with selected refreshed profile fields
/api/auth/logout	POST	Expire the platform session cookie
/api/chat	POST, GET	Customer chat through server-sent events; customer history through JSON
/api/v1/chat	POST	Company Bearer key, message, optional verified customer ID and request ID -> JSON chat result
/api/escalation	GET, PATCH	Tenant admin queue and pending ticket decision
/api/escalation/[id]	GET, PATCH	Customer-owned or admin ticket status; admin decision adapter

Company administration

Endpoint	Methods	Contract
/api/admin/users	GET, POST, PATCH	List/sync customer profiles, create local user, change access state
/api/admin/documents	GET, POST, DELETE	List metadata, upload multipart file, delete tenant-owned document
/api/admin/api-keys	GET, POST, DELETE	List credentials, generate named key, revoke by record ID
/api/admin/settings	GET, POST	Read tenant settings and save validated company database configuration
/api/admin/widget	GET, POST	Read/save validated website assistant settings
/api/admin/customer-sync	GET, POST	Read/save source settings or run sync with action=sync
/api/admin/stats	GET	Company user/document/chat/escalation counts
/api/admin/analytics	GET	Seven-day saved chat timeline
/api/admin/agents	GET	Company action log and aggregate action counts
/api/admin/reports/billing	GET	CSV of API request counts and saved chats
/api/admin/reports/audit	GET	CSV of escalation history

Widget and demonstration

Endpoint	Methods	Contract
/api/widget/config	GET	Public branding after company/origin validation
/api/widget/bootstrap	POST	Anonymous company/origin token
/api/widget/session	POST, GET	Company backend creates personal token; token holder validates current context
/api/widget/chat	POST	Widget Bearer token and chat input -> JSON result
/api/widget/escalation	GET	Token-protected customer-owned review status
/widget/frame	GET	Hosted assistant HTML and iframe security headers
/api/demo/login	POST	Prepared fictional customer selection -> platform cookie
/api/demo/enterprise-chat	POST	Nova customer session -> server-side enterprise API request
/api/demo/widget-token	POST	Nova demonstration token for an approved fictional identity

Operations and other routes

Endpoint	Methods	Current purpose
/api/ops/admins	GET, POST, PATCH	List, provision, and enable/disable business admins
/api/ops/agents	GET	Platform-wide action log
/api/ops/stats	GET	Platform record totals with simulated trend and revenue estimate
/api/ops/reports	GET	Placeholder returning OK
/api/health	GET	Application process response and timestamp
/api/agent	GET, POST	Legacy agent settings API; not integrated into the main chat policy
/api/kb	POST	Legacy direct vector upload path
/api/traces	GET	Legacy trace/history reader
/api/report	GET	PDF retrieval utility by report ID
/api/fleet	GET	Mock fleet data
/api/architecture	GET	Mock infrastructure health data
/api/seed	GET	Admin-only sample customers/sales seeding
/api/dev-seed	GET	Destructive development reset; blocked when NODE_ENV is production

The legacy agent, knowledge, trace, and report routes have material authentication gaps described later. Their presence in the route table should not be presented as complete production APIs.

Example enterprise request and response

The trusted client backend sends a Bearer company key and a body containing message, customerId, and requestId. A successful order result includes a reply string, actions containing queryDatabase, sources, requestId, and an orders array. A manager-review result includes escalationId. A public policy result may omit customerId, but personal workflows require it.

The server verifies the customer-to-company relationship. It does not accept a tenantId in place of a valid company key. There is no streaming enterprise response, conversation storage API, webhook system, or payment command in this version.

15 Pages and navigation

Public routes are /, /login, /docs, /architecture, /security, and /pricing. The pricing page explicitly says subscriptions are outside the prototype. /shader is a visual experiment. /walkwave and /nova are the fictional company demonstrations.

Company routes are /admin/login, /admin/dashboard, /admin/users, /admin/documents, /admin/api-keys, /admin/agents, /admin/analytics, /admin/reports, /admin/escalations, and /admin/settings. Operator routes are /ops/login, /ops/dashboard, /ops/admins, /ops/agents, /ops/reports, and /ops/system. Customer routes are /portal/register, /portal/login, /portal/chat, and /portal/history.

Older /studio, /launch, /replay, /incident, and /escalation page implementations are stored under `_legacy_routes` rather than `app`. They are not the active pages listed by the current production build. Some old navigation components and setup documentation still refer to them.

The active UI uses a shared authentication context, role-specific sidebars, responsive utility styles, externally loaded Google fonts/icons, and storefront CSS. Browser visuals, keyboard navigation, assistive technology behaviour, and mobile layout still need a dedicated presentation rehearsal; a successful build alone does not validate all of those interactions.

16 Runtime configuration and deployment

The project scripts provide `npm run dev`, `npm run build`, `npm run start`, and `npm run lint`. Demo commands include `demo:dev`, `demo:setup`, `demo:seed-customers`, `demo:verify`, `demo:verify-widget`, `demo:verify-customer-sync`, and `demo:widget-site`.

Configuration group	Variable names	Purpose
Platform Firebase	NEXT_PUBLIC_FIREBASE_API_KEY, AUTH_DOMAIN, PROJECT_ID, STORAGE_BUCKET, MESSAGING_SENDER_ID, APP_ID with the NEXT_PUBLIC_FIREBASE_ prefix	Initialize the platform Firebase project
Session security	JWT_SECRET	Sign cookie and widget tokens
Operator login	PLATFORM_OPS_EMAIL, PLATFORM_OPS_PASSWORD	Override the operator login defaults
AI and retrieval	GEMINI_API_KEY, GEMINI_MODEL, PINECONE_API_KEY, PINECONE_INDEX_NAME	Generation, embeddings, and vector index
Email	SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, SMTP_FROM, DEMO_EMAIL_TO	SMTP and demo recipient configuration
Application URLs	NEXT_PUBLIC_APP_URL, DEMO_API_BASE_URL, VERCEL_PROJECT_PRODUCTION_URL	Report links and Nova's backend API destination

Configuration group	Variable names	Purpose
Demo accounts	NOVA_DEMO_API_KEY, NOVA_ADMIN_PASSWORD, W ALKWAVE_ADMIN_PASSWOR D	Seeded demonstration integrations and managers
Optional AWS helpers	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION, CLOUDWATCH_LOG_GROUP, DYNAMODB_TRACES_TABLE, SQS_ESCALATION_QUEUE_U RL	Supporting infrastructure modules

The local configuration contains variable names for Firebase, Gemini, Pinecone, SMTP, demo accounts, JWT, and optional AWS services. Presence of a value does not establish that the credential is valid or that the corresponding service is connected. Secrets are excluded from this document.

The Next.js configuration externalizes PDF-related packages and includes PDF worker/canvas files in the document route's deployment trace. Chat, document upload, customer sync, enterprise chat, and widget chat routes set a maximum duration of 60 seconds where defined. Actual hosting limits still depend on the deployed environment.

Nova's custom backend uses DEMO_API_BASE_URL first, then a production Vercel URL when available, then localhost. A production deployment must not retain a localhost API destination. Firebase permissions and required query indexes must be provisioned outside the application; versioned rules and index configuration are not included in my-app.

The .env files are ignored by Git. Generated output, node_modules, and .vercel state are also ignored. There is no application-specific CI workflow, container deployment recipe, or infrastructure-as-code deployment in this folder. The README remains largely the original Next.js starter guide.

17 Implemented capabilities and honest boundaries

Capability	Accurate description
Policy answers	Implemented using indexed company text with Gemini and Pinecone retrieval/fallback
Personal order lookup	Implemented with tenant/customer checks and configured collection restrictions
Website widget	Reusable installation script and hosted iframe are implemented
Custom client interface	Enterprise API and Nova server-side demonstration are implemented
Email updates	SMTP workflow and request-ID duplicate prevention are implemented; inbox delivery needs confirmation
Manager approval	Persistent ticket and decision workflow is implemented; approval does not execute a refund
Customer sync	Selected profile import and reconciliation are implemented; it is not a continuous background service
Generic database agent	Not implemented; arbitrary collections, schemas, and SQL queries are not supported
Report agent	PDF utility exists, but the active chat workflow does not authorize or dispatch report creation
Decision trace pipeline	Logger utilities exist, but the main support request does not write full step-by-step traces
Guardrail pipeline	Regex guardrail module exists, but it is not invoked by the active chat service
AWS processing pipeline	Helper modules exist; they are not connected to the active chat flow
Infrastructure health	Operations health screen and architecture API show sample values
Revenue and billing	Subscriptions/payment billing are not implemented; operations MRR is calculated from tenant count
Enterprise sandbox	No actual microVM, gVisor, or arbitrary code execution sandbox is implemented here

The operator usage trend is randomly distributed from total invocation counts rather than calculated from timestamped requests. Operator "active end users" counts all user documents. Operator total invocations adds API usage counts and saved chats, which can count the same identified enterprise request twice. Admin recorded chat analytics are based on real persisted chat records, but they do not represent every attempted request.

Some screens contain outdated language. Website Agent says a general-purpose script is not implemented even though widget.js now exists. Operations pages refer to older Gemini/Vertex/Cloud Run architecture

names. The setup guide contains legacy route links. These should be corrected before presenting the documentation as fully aligned with the current application.

18 Security and engineering limitations to explain accurately

The project contains real ownership checks in the main support flows, but it is not ready to claim complete production security or formal compliance. The following issues are directly relevant to technical questions.

Authentication and tenant access

The legacy `/api/agent` endpoint accepts caller-supplied tenant IDs and writes configurations without a session check. `/api/kb` accepts direct vector uploads without an application authentication check. `/api/traces` reads records without authentication. `/api/report` returns a stored PDF by ID without checking tenant or user ownership. Older PDFs under `public/reports` are also static public assets.

API key deletion checks that the caller is an admin, but does not verify that the target key belongs to the admin's tenant before deleting it. Most admin endpoints trust the session role; only some recheck the live admin account. Disabling an admin therefore does not reliably revoke all existing session privileges immediately. Company API keys are not automatically disabled with the company admin account.

Cookie sessions and middleware fall back to a fixed development signing secret if `JWT_SECRET` is absent, including in production. Widget signing separately fails closed in production without a secret. Operator login has hardcoded default credentials. These defaults should be removed or restricted before real deployment. Local operator environment overrides were not present in the reviewed configuration names.

Database permissions and privacy

The repository setup guide includes `allow read, write: if true as a demo Firestore rule`. Deployed rules were not inspected. This is a deployment risk to verify, not evidence that every production Firebase project is currently open. Because custom platform cookies are separate from Firebase Auth, the database authorization architecture needs a deliberate production design rather than relying only on route-level checks.

Input PII scrubbing and regex injection detection exist in `lib/guardrails.ts`, but the active shared service does not call them. The model receives user input/history for policy answers, and original messages can be persisted. There is no evidence here of automated PII removal, certified compliance, encryption implemented by the application for stored API keys, or a retention policy.

Reliability and lifecycle

Escalation decisions check pending status and then update the document without a transaction. Two concurrent decisions can both pass the initial check and overwrite one another. The widget status endpoint reads `decisionNote`, while the admin saves `adminNote`, so the reusable widget can miss the manager's note even while displaying the correct status.

Customer sync locks and batches reduce duplicate work, but a failure after one batch can leave partial reconciliation. Sync depends on an admin page or an explicit request. Source outages leave previous accounts available. Disabling the widget rechecks on subsequent requests; there is no general platform-wide revocation/version mechanism for every cookie session.

Several collection reads load all matching records and sort/filter in memory. History, action logs, documents, operator totals, and parts of the escalation queue have limited or no server-side pagination. Current limits and the small demonstration dataset make the workflow manageable, but large-company scale has not been demonstrated.

There is no rate limiting for the main portal chat, enterprise chat, or password login endpoints. Widget rate limits rely on the forwarded client IP header and the hosting environment's handling of that header. End-to-end cost controls, per-key quotas, model abuse limits, and global monitoring remain future work.

Development tools and test maintenance

`/api/dev-seed` clears multiple platform collections without login when running outside production. It is blocked in production mode, but a development server connected to real data remains risky. `/api/seed` mutates demo data through GET. Setup scripts also create or overwrite records; they are not passive health checks.

The customer sync verification script creates a test tenant without listing its temporary source collection in `allowedCollections`. With the new collection restrictions, enabling that source will be rejected unless the test setup is updated. The demo setup script writes `allowedCollections=demo_orders` and can remove a previously configured sync source from the saved allowlist. These are configuration/test-maintenance gaps, not permission-check failures.

19 Local evidence from this review

The production build completed successfully, including TypeScript checking and static page generation. The build reported the middleware file convention as deprecated in favour of proxy. The isolated database access verification script passed parsing, exact matching, configured order lookup, blocked collection handling, and company identity checks using mocked order dependencies.

Read-only HTTP checks against a local production server returned 200 for the public pages, Walkwave, and Nova. Protected admin, operator, and customer chat pages returned 307 redirects to their login routes. Main admin/customer/enterprise/widget APIs returned 401 without the required session, key, or token. The production-mode development reset endpoint returned 403. The unauthenticated legacy trace reader returned 200 for a nonexistent trace ID, and the operations report endpoint returned 200 with only OK.

The full npm run lint check failed with 334 errors and 4,068 warnings. Of those, 212 errors and 4,036 warnings came from generated files inside a nested `my-app/.next` directory. Active app files still had 45 errors and 27 warnings, and lib files had eight errors and one warning. The result identifies both a generated-output exclusion problem and source issues; it should not be presented as thousands of independent functional defects.

Live AI answers, document upload/indexing, sync reconciliation, SMTP delivery, manager decisions, and authenticated browser workflows were not rerun during this review. Existing integration scripts exercise those paths but write connected records, and one optional flag sends email. Their presence shows intended verification coverage; it does not establish that all live integrations passed today.

20 Design decisions and tradeoffs

One shared support service

The same support behaviour serves the portal, custom enterprise interface, and reusable widget. This reduces the chance that one channel applies a different order ownership rule or email workflow. The tradeoff is that a shared-service issue affects all channels, and the current service combines routing, retrieval, action orchestration, and persistence in one module.

Deterministic operational data

Order answers use actual database fields and qualifying refund review uses server-side logic. The language model handles policy wording while operational facts and side effects stay under application control. Keyword intent detection is easy to inspect, but it has limited coverage for unusual phrasing, multilingual questions, and custom order ID formats.

Stored text alongside vectors

Firestore retains indexed policy text so that retrieval can continue during a vector search problem. This improves demonstration resilience, but it duplicates sensitive business content and can supply a larger, less selectively relevant prompt. A production design needs document access lifecycle, retention, relevance thresholds, and evaluation.

Profile import instead of password migration

The company keeps its own authentication, while AgentForge imports a limited profile and identity mapping. That avoids copying passwords and supports customer-owned orders. The client backend must still securely map its verified identity to the correct platform user ID, and reconciliation needs a reliable background mechanism at larger scale.

Human review before financial execution

The prototype stops at a persisted manager decision. It demonstrates control over high-value support requests without connecting a payment system. A production refund integration would need separate authorization, amount and currency validation, transaction identifiers, idempotency, failure reconciliation, and an audit trail.

21 Question and answer preparation

The answers below describe the current implementation. Use them as a factual starting point and adapt the wording to your own presentation.

1 What does AgentForge do

AgentForge adds company-specific customer support to a website through a ready-made widget or a custom interface. It answers from uploaded policies, checks authenticated customer orders, sends requested updates by email, and creates review tickets for qualifying refund requests. Company admins manage the content, users, integrations, and decisions.

2 Why is this more than a general chatbot

It connects conversational support to company knowledge and authenticated operational data. A general policy answer comes from company documents, while an order answer comes from a tenant- and customer-filtered database query. It also records real email and manager review actions rather than merely describing them in text.

3 What is the strongest working demonstration

A customer asks for the return window, checks a personal order, requests the shipping update by email, then asks for a LKR 75,000 refund. The manager reviews the persisted ticket and rejects or approves it. Nova also demonstrates the same backend through a company server calling the enterprise API.

4 Is this a fully autonomous multi-agent system

The current implementation is a coordinated support service with functions for database lookup, email, and review. The server routes the main operational requests. The model can support declared tool calls in a generic helper, but the active policy-answer path disables tools. Independently running agents and a general autonomous workflow engine are future work.

5 How do you prevent the model from inventing order information

The server retrieves customer-owned order records and constructs the shipping answer from those fields. It checks tenant membership and active status before querying. The model is not responsible for generating the order status or tracking number in the main support path.

6 How are policy answers grounded

Uploaded policy text is chunked and embedded. Pinecone searches for relevant chunks with a company filter, and the model receives the selected text with instructions to answer only from it. Retrieved source titles are returned. This reduces unsupported answers, but model accuracy still requires evaluation and the current citations are document-level.

7 What happens if Pinecone is unavailable

Retrieval falls back to real indexed document text stored in Firestore, with a 20,000-character text budget. If no indexed company text is available, the assistant says it lacks the document. An upload cannot be marked indexed unless its embedding and vector upsert succeed.

8 Which model do you use and why

The source defaults to gemini-flash-lite-latest for text and gemini-embedding-2 for embeddings. The text model is configurable through GEMINI_MODEL. The code expresses a lightweight generation choice, but there is no measured comparison proving it is the cheapest or most accurate model for every company.

9 Can the system learn directly from any database

It can connect to another Firebase Firestore project, but the supported readers are narrow. Orders need the expected tenant/customer fields and schema; profile sync imports selected fields. Arbitrary SQL, collection-wide conversational queries, automatic schema discovery, and field mapping are not implemented.

10 How do you control which collections are accessible

The admin saves an exact allowlist. The order reader and customer sync source must both be listed. Empty lists block those company reads, and unlisted names are denied. The control is application-level; the company's deployed Firestore rules still determine what the SDK can access.

11 Does a Firebase config file give unrestricted access

No. It identifies a Firebase application and project. The SDK still operates under the project's deployed permissions. This prototype does not turn that web configuration into a service-account identity or rewrite the project's rules.

12 How is one customer's order kept private

The server authenticates a platform user, validates tenant membership and active state, then queries orders using tenantId and customerId together. Imported users are mapped to their original company customer IDs. Another customer's order ID cannot change that ownership filter.

13 What is a tenant

A tenant is the company boundary represented by a tenant ID. Admins, customers, policies, keys, settings, and actions are associated with it. Main support APIs derive the tenant from the authenticated session, API key, or signed widget token.

14 Can every customer use the public widget

Anonymous visitors can ask policy questions. Personal orders and actions need a verified customer token. The client's backend verifies the website login, supplies the mapped active AgentForge user ID, and requests a short-lived widget session using its company API key.

15 Why use an iframe for the widget

It keeps the assistant interface hosted by AgentForge while the business website installs a small script. Shadow DOM isolates launcher styling, and the iframe has an origin-based framing policy. Token authentication avoids depending on third-party platform login cookies.

16 What is the difference between widget and enterprise API

The widget supplies the interface and hosted chat panel. The enterprise API supplies the support response for an interface the company builds itself. Both call the shared chat service, so order lookup, policy retrieval, email, and review behaviour are reused.

17 Where does the company API key live

It must remain on the company's backend. The public installation snippet only includes the service URL and public company ID. The current platform stores keys in Firestore as plaintext; production hardening should hash them and improve management and revocation controls.

18 Are customer passwords copied during sync

No. Sync imports selected identity and profile fields and keeps the original external customer mapping. Passwords and password hashes are excluded. Imported customers sign in through their company website, which requests the corresponding AgentForge token.

19 Is sync continuous

It runs when Users and Access requests the user list or when an admin clicks Sync now. The service normally throttles successful automatic sync to about 30 seconds, and supports up to 500 source profiles. A scheduled background worker or realtime subscription is not implemented.

20 What happens when a source customer is removed

After a successful sync, a missing or inactive source profile loses effective platform access. An admin disable also survives later sync. A source outage leaves the last imported state in place and reports an error, so production use needs a deliberate stale-access policy.

21 How does email sending avoid duplicates

The caller supplies or receives a request ID. The server transactionally claims the tenant/customer/request combination before SMTP sending. A repeated ID does not resend. A successful SMTP response means server acceptance, and inbox delivery must be verified separately.

22 Can the assistant email any person

The current workflow requires an identified active customer and an explicit email request. It accepts an address in the current message or uses the customer's configured address. It is not a bulk marketing system. Recipient policy, abuse limits, and confirmation rules would need strengthening for production.

23 When is a refund escalated

The demo rule escalates refund/reimbursement messages above LKR 50,000, foreign-currency refund requests, and refund messages requesting a manager or human. It creates a pending ticket. The rule is implemented on the server, so it does not depend solely on the model choosing an escalation tool.

24 Does approval automatically refund money

No. Approval records the manager's permission and note. There is no payment processor integration or refund execution. The interface explicitly tells the customer that no payment was executed in this demonstration.

25 Does every request for a human create a ticket

Not currently. The active deterministic rule checks refund or reimburse first. General requests for a human need a broader handoff route or explicit intent handling. It is more accurate to describe the present feature as qualifying refund review rather than universal escalation.

26 Can admins see customer questions and decisions

Admins can inspect company escalation requests with customer identity, original message, rule, status, and decision notes. Saved chat history exists for identified users, while admin metrics count those records. The application does not expose a complete searchable conversation audit console for every request.

27 Do you have full decision trace auditing

Trace logger utilities and a legacy trace reader are present, but the active chat service does not call the complete trace pipeline. It records chat history, actions, email request state, and review tickets. Full per-step tracing and durable external audit integration remain incomplete.

28 Are the guardrails active

The code contains regex input and output guardrail functions, but the main chat flow does not invoke them. Active controls include identity checks, scoped order queries, allowed collections, restricted policy prompts, disabled policy tools, explicit email intent, and server-side refund review. We should not claim automatic PII scrubbing or a complete three-layer guardrail pipeline.

29 Is the system production ready

The main support prototype builds and has useful access checks, but production readiness requires work. Priority gaps include legacy route authentication, key deletion ownership, consistent account revocation, secure defaults, deployed database rules, rate limits, background sync, and real monitoring. The correct claim is a working prototype with identified hardening needs.

30 Are all dashboard numbers real

Some admin metrics come from Firestore counts and saved chats. Operations trend, MRR, infrastructure health, and static reports include simulated or derived values. They must be labelled accordingly. The application has not measured true model token cost, uptime, revenue, or enterprise-scale latency.

31 Why do you keep Firestore and Pinecone

Firestore stores application records and indexed document text. Pinecone provides vector similarity search over chunks. Keeping text also supports a fallback when vector retrieval fails. The tradeoffs are duplication, retention requirements, and the need to keep vector/document deletion consistent.

32 What are the file upload limits

Non-empty PDF and TXT files are accepted up to 4 MB, with up to 80,000 extracted characters. PDFs need selectable text. OCR, image extraction, Office documents, and arbitrary file types are outside the current ingestion capability.

33 How would you scale to more companies

First close authentication and tenant ownership gaps. Then add indexed queries and pagination, background sync, per-key quotas, reliable auditing, document lifecycle controls, and measured model/retrieval performance. The present 500-customer sync limit and in-memory list processing are prototype constraints, not demonstrated large-scale capacity.

34 What happens if the AI service fails

Policy generation can fail or time out, and the API shows a service error rather than inventing a successful answer. Order lookup can operate without generation because its reply is deterministic, provided Firestore is available. Pinecone has a stored-text fallback; Gemini generation itself has no second-provider fallback.

35 How is the operator different from a company admin

The operator provisions and manages business admin accounts and can read platform-wide activity. A company admin configures and manages its own company support setup. An ordinary customer can use support and see their own history or review status, not company administration.

36 How do you demonstrate that customers cannot share orders

Use two separate customer identities. Jane should retrieve WW-1001 and Bob should retrieve WW-1002. Asking for Bob's order while using Jane's authenticated identity should return no matching order. The same principle applies to Alice and Sam in Nova and to cross-company API identity checks.

37 Are these real shop purchases and courier updates

No. The demonstration writes fictional orders to Firestore and genuinely retrieves those records. Shipping statuses and tracking references are sample data. There is no live courier, inventory reservation, checkout, or payment integration.

38 What has been verified in this review

The current production build and TypeScript step passed, isolated database access checks passed, public pages responded, and key anonymous access barriers behaved as expected. The full lint check failed and identified both source issues and generated-output noise. Live AI, email, uploads, sync, and authenticated end-to-end flows still need a presentation rehearsal.

39 What should not be claimed during the presentation

Avoid claiming certified compliance, production security, automatic refunds, continuous background sync, generic database reasoning, measured financial savings, live infrastructure health, a functioning microVM sandbox, or a fully connected AWS audit pipeline. Explain the implemented support workflow and the planned extensions separately.

40 What is the next development priority

The first priority is closing authorization and secret-default gaps and aligning database permissions with the authentication design. The next priorities are reliable lifecycle handling, background sync, rate and cost controls, real monitoring, and broader verification. Additional connectors and autonomous workflows should follow those foundations.

22 Suggested presentation walkthrough

Start by explaining the company/customer problem and the two integration modes. Show Company Settings, including the Firebase connection, allowed collections, order collection, and profile source. Explain that access is explicit and schema-specific.

Show an indexed company policy and ask the storefront assistant for the return window. Highlight the document source. Then ask for a personal shipping update and explain that the server reads authenticated order data. If SMTP has been rehearsed, request one update by email and show its received copy, distinguishing acceptance from delivery.

Ask for a LKR 75,000 refund. Switch to the manager browser profile, open the pending queue, inspect the original request and rule, and save a decision. Return to the customer and show the updated status. State clearly that the decision does not move money.

Finish with Nova's custom interface or a prepared enterprise API request. Explain that the company backend keeps the API key and supplies a verified customer mapping. Close with the production hardening priorities and the next integrations, rather than promising capabilities that are still represented only by helper code.

23 Glossary

Term	Meaning in this project
Tenant	A company boundary represented by tenantId
RAG	Selecting stored company knowledge before language-model answer generation
Embedding	A numeric representation of text used for similarity search
Vector database	Pinecone's storage and search service for policy embeddings
JWT	A signed token used for platform or widget identity
Idempotency	Avoiding repeated execution when the same request ID is submitted again
SSE	Server-sent events used by the portal/Walkwave chat endpoint
Allowlist	Exact collection names that supported company readers may access
Origin	Scheme, host, and port of an allowed embedding website
Reconciliation	Comparing source customer profiles with previously imported platform identities
SMTP	The email server protocol used to send requested updates
Human review	Persisting a request for an admin decision without executing a payment

24 Source reference map

The technical statements in this guide are anchored to the following application files and groups at the stated baseline.

- package.json, next.config.ts, tsconfig.json, middleware.ts: stack, build, runtime, and page protection.
- lib/chat-service.ts and lib/tools.ts: support intent routing, order ownership, email, persistence, and review creation.
- lib/knowledge.ts and lib/rag.ts: extraction, chunking, embedding, retrieval, fallback, model selection, and allowed tool dispatch.
- lib/session.ts and app/api/auth: cookie sessions, account creation, password checks, and operator login.
- lib/company-database.ts and lib/database-access.ts: external project selection and exact collection restrictions.
- lib/customer-sync.ts and app/api/admin/customer-sync: imported identity, batching, locks, and source reconciliation.
- lib/widget-auth.ts, lib/widget-config.ts, app/api/widget, app/widget/frame, and public/widget: widget security and runtime.
- app/api/v1/chat and app/api/demo: enterprise and fictional storefront integration.

- `app/admin`, `app/portal`, and `app/ops`: displayed features, metrics, admin decisions, and client response handling.
- `lib/guardrails.ts`, `lib/trace-logger.ts`, `lib/pubsub.ts`, `lib/cloud-logging.ts`, `lib/bigquery.ts`, and `lib/secret-manager.ts`: supporting capabilities whose active integration is limited or absent.
- `app/api/agent`, `kb`, `traces`, `report`, `fleet`, `architecture`, `seed`, and `dev-seed`: legacy routes, utilities, and identified limitations.
- `scripts/setup-demo.cjs`, `scripts/verify-demo.cjs`, `scripts/verify-widget.cjs`, `scripts/verify-customer-sync.cjs`, `scripts/verify-admin-demo.cjs`, and `scripts/verify-database-access.cjs`: setup and verification intent.
- `FIREBASE_SETUP.md`, `README.md`, `survival_plan.md`, and `_legacy_routes`: historical setup guidance and earlier application scope.